

Wojciech Cieślik, Michał Świątek
Informatyka rok II

gr. 6, Pon. godz. 16:45 B

Lokalizacja punktów w przestrzeni dwuwymiarowej. Metoda trapezowa.

Algorytmy Geometryczne 2025/2026

Spis treści

1. Wstęp	2
2. Opis teoretyczny algorytmu	2
2.1. Opis problemu	2
2.2. Podział trapezowy	2
2.3. Struktura mapy trapezowej	2
2.4. Struktura przeszukiwań	2
2.5. Opis algorytmu	3
3. Dokumentacja	3
3.1. Część użytkownika	3
3.2. Część techniczna	4
3.2.1. Moduły wykorzystywane w programie:	4
3.2.2. Struktury danych:	4
3.2.3. Funkcje zaimplementowane w programie:	4
3.2.4. Funkcje pomocnicze do wizualizacji:	5
4. Testy i wizualizacja	6
4.1. Dane techniczne	6
4.2. Testy wizualizacyjne	6
4.2.1. Zbiór A	7
4.2.2. Zbiór B	7
4.2.3. Zbiór C	8
4.2.4. Zbiór D	8
4.3. Testy wydajnościowe	8
4.3.1. Czas algorytmu tworzenia mapy trapezowej	9
4.3.2. Czas lokalizacji punktu na mapie	10
4.3.3. Rozmiar struktury przeszukiwań	11
5. Podsumowanie	11
Literatura	11

1. Wstęę

Celem projektu było stworzenie programu rozwiązującego problem lokalizacji punktu w przestrzeni dwuwymiarowej z wykorzystaniem metody trapezowej. W tym celu zaimplementowana została funkcja budująca mapę trapezową oraz graf przeszukiwań dla zadanego podziału przestrzeni. Dodatkowo działanie programu jest wizualizowane wykorzystując dodatkowe funkcje.

2. Opis teoretyczny algorytmu

2.1. Opis problemu

Niech S oznacza podział przestrzeni dwuwymiarowej zawierający n odcinków. Zapytanie o lokalizację punktu w S oznacza znalezienie takiej ściany f zawartej w S , w której znajduje się zadany punkt. Algorytm ma na celu stworzenie takiej reprezentacji S , która pozwala odpowiadać na zapytanie lokalizacji punktu w jak najmniejszym czasie, zajmując także jak najmniej miejsca w pamięci.

2.2. Podział trapezowy

Metoda trapezowa polega na stworzeniu mapy trapezowej $T(S)$ dla danego zbioru S zawierającego n odcinków reprezentującego podział przestrzeni. Wraz z mapą $T(S)$ algorytm stwarza też strukturę przeszukiwań D pozwalającą odpowiadać na zapytanie o lokalizację punktu. Tworzenie mapy trapezowej następuje w sposób przyrostowy poprzez dodawanie kolejnych odcinków z S i tworzenie pionowych przedłużeń wychodzących z ich wierzchołków kończących się po napotkaniu innego odcinka. Kolejność dodawania odcinków z S wpływa na wielkość i czas odpowiedzi na zapytania struktury D , jednak, wykorzystując randomizację, założyć można,^[1, s. 154-164] że złożoność pamięciowa struktury D jest rzędu $O(n)$, a czas odpowiedzi na zapytanie rzędu $O(\log n)$. Ostatecznie, randomizowany algorytm przyrostowy budujący mapę $T(S)$ i strukturę D ma oczekiwaną złożoność obliczeniową $O(n \log n)$.

Podział trapezowy przyjmuje dla zbioru S następujące założenia:

- Odcinki nie przecinają się
- Wierzchołki odcinków mają parami różne współrzędne x .

Zbiór S spełniający powyższe założenia jest w położeniu ogólnym. Dokonując przekształcenia ścinającego na odcinkach z S o dostatecznie mały kąt φ oraz stworzenia mapy trapezowej dla nowo powstałego zbioru φS , który już jest w położeniu ogólnym. Okazuje się jednak,^[1, s. 164-168] że przekształcenie to może pozostać jedynie w domyśle i przy sprawdzeniu czy dany punkt leży na lewo od drugiego wystarczy porównać je leksykograficznie.

2.3. Struktura mapy trapezowej

Mapa trapezowa $T(S)$ reprezentowana jest jako graf trapezów i trójkątów interpretowanych na potrzeby programu jako zredukowane trapezy. Każdy stworzony trapez ma dwa boki pionowe (w przypadku trójkątów uznajemy także pionowe boki o długości zerowej). Trapezy sąsiadują ze sobą, gdy mają wspólną ścianę pionową. Każdy trapez ma maksymalnie czterech sąsiadów. Mają też wskaźnik do swojego węzła w strukturze przeszukiwań D .

2.4. Struktura przeszukiwań

Struktura przeszukiwań D reprezentowana jest przez DAG - acykliczny graf skierowany przypominający drzewo binarne. Występują w niej trzy typy węzłów:

- Węzeł trapezu zawierający wskaźnik do trapezu w mapie trapezowej, stanowią liście grafu.

- Węzeł wierzchołka zawierający wskaźnik do jednego z wierzchołków odcinków z S oraz wskaźniki do dwóch węzłów dzieci. W węzłach wierzchołków testujemy, czy zadany punkt leży na lewo czy na prawo od trzymanego wierzchołka. Jeżeli leży na lewo od danego wierzchołka, przechodzimy do lewego dziecka, a jeżeli na prawo od wierzchołka, do prawego dziecka.
- Węzeł odcinka zawierający wskaźnik do jednego z odcinków z S oraz wskaźniki do dwóch węzłów dzieci. Analogicznie do węzłów wierzchołka, idziemy do lewego dziecka jeśli punkt leży nad odcinkiem i do prawego, jeśli leży pod nim.

2.5. Opis algorytmu

Kolejne kroki algorytmu przebiegają następująco:

1. Tworzymy prostokąt zawierający wszystkie odcinki w S i inicjalizujemy nim strukturę mapy trapezowej $T(S_0) = T(\emptyset)$.
2. Tworzymy listę $S_n = (s_1, s_2, \dots, s_n)$ będącą losową permutacją odcinków ze zbioru S .
3. Dla każdego $i = 1, 2, \dots, n$ powtarzamy:
 - (a) Znajdujemy na obecnej mapie $T(S_{i-1})$ trapezy $\Delta_1, \Delta_2, \dots, \Delta_k$ przecięte przez odcinek s_i .
 - (b) Usuwamy z $T(S_{i-1})$ trapezy $\Delta_1, \Delta_2, \dots, \Delta_k$ zamieniając je na trapezy powstałe po dodaniu s_i . Otrzymujemy w ten sposób mapę $T(S_i)$.
 - (c) Usuwamy z D_{i-1} węzły odpowiadające trapezom $\Delta_1, \Delta_2, \dots, \Delta_k$ i dodajemy węzły dla nowych trapezów odpowiednio łącząc je węzłami wierzchołków i odcinków.
4. Po dodaniu wszystkich odcinków, otrzymujemy struktury $T(S_n)$ i D_n .

Krok 3.(a) wykonujemy z wykorzystaniem połączeń sąsiedzkich trapezów w mapie. Najpierw znajdujemy trapez Δ_1 , wykonując zapytanie na D_{i-1} dla punktu będącego lewym wierzchołkiem odcinka s_i . Następnie podążamy wzdłuż odcinka s_i przechodząc po trapezach, dodając do listy przeciętych trapezów odpowiednich sąsiadów na mapie. Kończymy po napotkaniu prawego wierzchołka odcinka s_i .

Szczegóły implementacyjne algorytmu przedstawione są w kolejnej sekcji.

3. Dokumentacja

3.1. Część użytkownika

Program wykonać należy wewnątrz środowiska dostępnego w narzędziu przygotowanym przez studentów z koła naukowego Bit, dostępnego pod adresem:

<https://github.com/aghbit/Algorytmy-Geometryczne>

Program dostępny jest w pliku *cieslik_swiatek_kod.ipynb*. Przedstawiona jest w nim implementacja wszystkich funkcji potrzebnych do algorytmu. Dostępna jest też aplikacja graficzna wraz z funkcjami pomocniczymi pozwalającą na tworzenie testów oraz funkcje pozwalające na testy czasowe programu. Folder *tests* zawiera przygotowane przykłady testowe programu.

Główna funkcja odpowiedzialna za utworzenie struktur mapy trapezowej i struktury przeszukiwań to *build_trapezoidal_map* zwracająca korzeń grafu przeszukiwań przyjmującą listę odcinków reprezentowaną jako listę par punktów w położeniu ogólnym. Punkty reprezentowane są jako para zmiennych rzeczywistych odpowiadających położeniu punktu w przestrzeni.

W celu uruchomienia programu należy uruchomić kolejne moduły w pliku *cieslik_swiatek_kod.ipynb*.

Moduł z linijką *start_generator()* uruchamia aplikację graficzną. Otwiera się wówczas osobne okno. Za pomocą myszki możliwe jest zadawanie kolejnych odcinków poprzez kliknięcie dwóch punktów na mapie. Dodawane są jedynie te odcinki, które są w położeniu ogólnym. Program umożliwia także wyczyszczenie wstawionych już segmentów za pomocą przycisku *Wyczyść*. Opcja *Generuj* odpala

dodatkowe okno z miejscem na wprowadzenie danych, następnie losowo generując podaną liczbę odcinków na danym obszarze. Kolejno opcje *Zapisz* i *Wczytaj* pozwalają na odczyt i zapis plików .json przechowujących zbiór odcinków. Zamykając okno generatora wykorzystując symbol „X” w prawym górnym rogu ekranu ostatni zaznaczony zbiór jest zapisywany w programie i wykorzystywany w pozostałych modułach.

W głównej części programu znajdują się przykłady wizualizacji zarówno budowania mapy trapezowej, jak i grafiki przedstawiające efekt wyszukiwania przez program położenia punktu.

3.2. Część techniczna

3.2.1. Moduły wykorzystywane w programie:

- numpy - moduł odpowiedzialny za operacje matematyczne.
- matplotlib - moduł obsługujący interfejs graficzny.
- Tkinter - moduł pomocniczy do interfejsu.
- random - moduł wykorzystany do permutowania zbioru odcinków
- json - moduł pozwalający na operację na plikach .json
- time - moduł pozwalający na pomiar czasu

3.2.2. Struktury danych:

- Point:

Klasa implementująca punkt w dwuwymiarowej przestrzeni - zawiera informację o współrzędnych punktu.

- Segment:

Klasa reprezentująca odcinek. W ramach konstruktora struktura układa końce odcinka w kolejności od lewej do prawej.

- Trapezoid:

Reprezentuje trapezy w mapie trapezowej. Określa granice trapezu, jego indeks, wszystkich sąsiadów oraz wskaźnik na liść w grafie przeszukiwań odpowiadający danemu trapezowi.

- Node:

Bazowa klasa węzłów. Przechowuje informacje o lewym i prawym dziecku i odniesienia do listy węzłów nadrzędnych. Posiada też metodę *replace_child*, która pozwala na zamianę dziecka.

- XNode:

Klasa pochodna klasy Node odpowiedzialna za wierzchołki odcinków.

- YNode:

Klasa pochodna klasy Node odpowiedzialna za odcinki.

- SinkNode:

Klasa pochodna klasy Node - liście struktury wyszukiwania. Posiada odnośnik do odpowiadającego danemu liściowi trapezowi w mapie trapezowej.

3.2.3. Funkcje zaimplementowane w programie:

- *is_up(segment, p)*:

Funkcja zwracająca prawdę w przypadku gdy punkt znajduje się powyżej danego segmentu. Obliczana jest z wykorzystaniem orientacji. Jest wykorzystywana w związku z węzłami YNode do podejmowania decyzji podczas przeszukiwania grafu.

- *locate(node, p)*:

Funkcja służąca do lokalizacji punktu na mapie trapezowej wykorzystując strukturę przeszukiwań.

- `init_tree(min_x, min_y, max_x, max_y)`:

Funkcja inicjalizująca mapę trapezową poprzez stworzenie ramy ograniczającej pokrywającej cały obszar przeszukiwania. Budowane są górny i dolny odcinek graniczny oraz punkty boczne, a następnie tworzony jest duży trapez. Funkcja zwraca korzeń struktury DAG, którym jest `SinkNode` powiązany z tym trapezem.

- `find_intersected_trapezoids(root, new_segment)`:

Funkcja wyszukuje wszystkie trapezy, przez które przechodzi nowo dodawany odcinek. Najpierw lokalizowany jest trapez zawierający lewy koniec odcinka, a następnie algorytm przechodzi do kolejnych trapezów po prawej stronie, korzystając z relacji sąsiedztwa. Proces kończy się po dotarciu do trapezu zawierającego prawy koniec odcinka. Wynikiem jest lista trapezów uporządkowana od lewej do prawej.

- `replace_node(old_node, new_sub_root, root)`:

Funkcja dla każdego rodzica starego węzła podmienia go na nowy pod-korzeń. Odwołania rodziców wskazujące na stary węzeł są przepinane na nowy węzeł. Umożliwia to na rozbudowę grafu wyszukiwania bez konieczności ponownego budowania grafu od podstaw.

- `link_left_neighbours(old_trap, new_trap)` i `link_right_neighbours(old_trap, new_trap)`:

Służą do przepisywania sąsiadów starego trapezu do nowego i aktualizacji wskaźników w trapezach.

- `handle_single_trapezoid(root, old_trap, seg)`:

Funkcja rozważająca przypadek odcinka przecinającego wyłącznie jeden trapez. Rozbija go na cztery nowe trapezy i podmienia jeden z liści w grafie.

- `handle_multi_trapezoid(root, traps, seg)`:

Funkcja odpowiedzialna za wstawianie odcinka przecinającego więcej niż jeden istniejący trapez. Funkcja dzieli każdy trapez przecięty przez odcinek na część górną i dolną, scala trapezy nad i pod odcinkiem tam gdzie to możliwe, aktualizuje sąsiedztwa trapezów i przebudowuje graf. Zwraca nowy korzeń struktury przeszukiwań.

- `add_segment(root, new_seg)`:

Funkcja pomocnicza dodająca odcinek do mapy - decyduje o wyborze funkcji do danego odcinka na podstawie liczby wykrytych trapezów przecinanych.

- `get_bounding_box(segments, margin=1.0)`:

Funkcja wyznacza obszar algorytmu - oblicza minimalne i maksymalne wartości współrzędnych x i y powiększone o dany margines. Dzięki temu początkowa rama ograniczająca obejmuje całą potrzebną przestrzeń.

- `build_trapezoidal_map(segments)`:

Funkcja odpowiedzialna za budowę mapy trapezowej. Tworzona jest kopia listy odcinków, która następnie jest permutowana w sposób losowy i następnie z wykorzystaniem powyższych funkcji tworzona jest mapa trapezowa i struktura przeszukiwań. Zwracany jest korzeń struktury przeszukiwań umożliwiający szybkie wyszukiwanie położenia punktów w podzielonej przestrzeni.

3.2.4. Funkcje pomocnicze do wizualizacji:

- `get_all_trapezoids(node, visited=None)`:

Funkcja zbierająca na listę wszystkie trapezoidy utworzone w mapie trapezowej.

- `get_y_on_edge(edge, x)`:

Funkcja zwracająca wartość y krawędzi w danym punkcie współrzędnej x -owej równej x .

- `draw_actual(root)`:

Rysuje pełną aktualną mapę trapezową na podstawie aktualnej struktury przeszukiwań i mapy trapezowej.

- `build_trapezoidal_map_visualization(segments_list)`:

Buduje mapę trapezową krok po kroku, wizualizując kolejne etapy jej konstrukcji.

- `graphical_result(segments_list, point)`:

Graficznie prezentuje wynik lokalizacji punktu z wykorzystaniem algorytmu.

4. Testy i wizualizacja

4.1. Dane techniczne

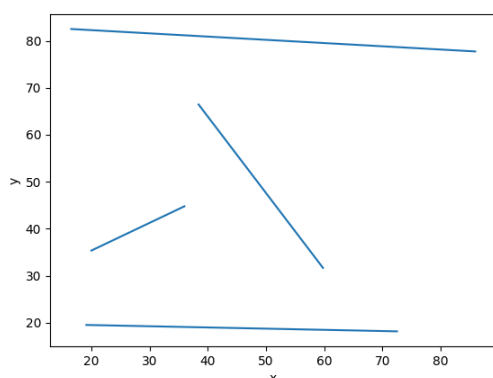
Wszystkie przedstawione poniżej testy zostały przeprowadzone na sprzęcie o poniższych danych technicznych:

- System operacyjny: Fedora Linux 43 (Workstation Edition) (x86-64)
- Procesor: Intel Core i7-12700H
- Pamięć RAM: 32 GB DDR4 3200 MHz
- Środowisko: Jupyter Notebook
- Język: Python 3.13.5

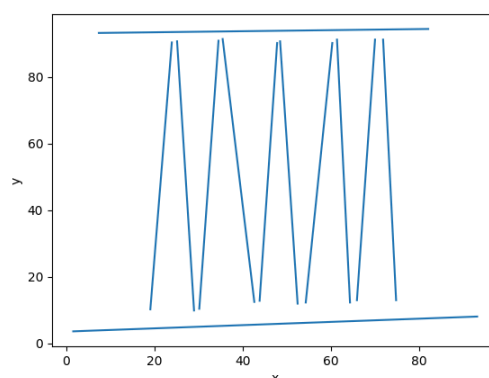
4.2. Testy wizualizacyjne

Do sprawdzenia poprawności oraz wizualizacji działania algorytmu utworzone zostały cztery zbiory odcinków przedstawione na Rysunkach 1, 2, 3 i 4. Kolejne sekcje zawierają wizualizację map trapezowych dla zbiorów A, B, C i D oraz krótki komentarz na temat poszczególnych zbiorów.

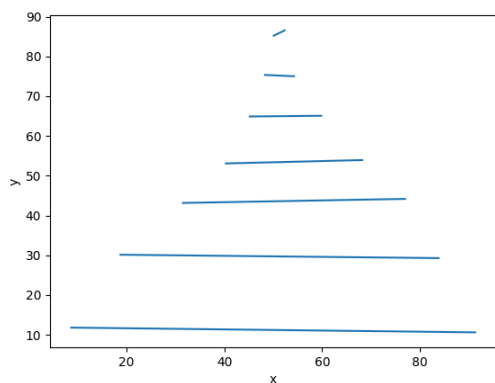
Animacje krokowe budowy mapy dla danych zbiorów dostępne są do wygenerowania w pliku notatnika *cieslik_swiatek_kod.ipynb*. Są także dostępne w postaci plików gif w załączonym folderze *res*. W pliku możliwe jest także wygenerowanie animacji dla zbioru zadanego przez użytkownika w czasie jego działania.



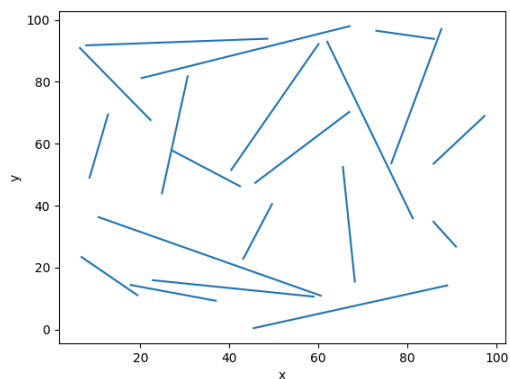
Rysunek 1: Zbiór A



Rysunek 2: Zbiór B



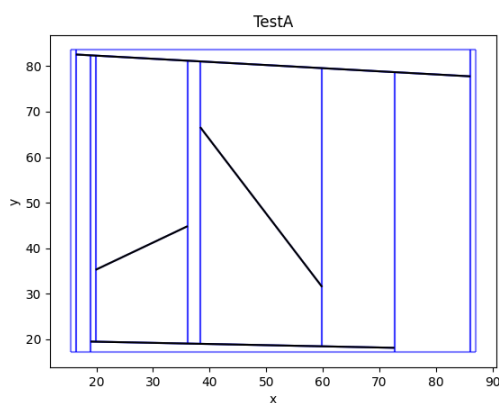
Rysunek 3: Zbiór C



Rysunek 4: Zbiór D

4.2.1. Zbiór A

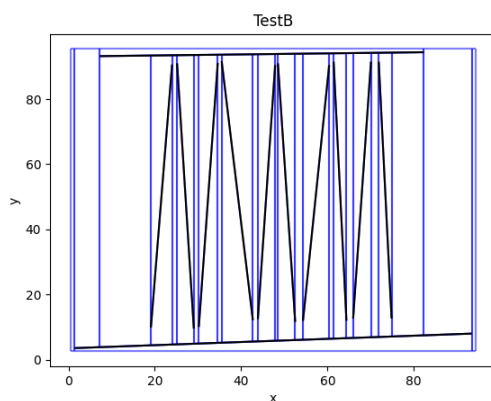
Pierwszy ze zbiorów przygotowany został w celu sprawdzenia czy algorytm poprawnie tworzy i scala trapezy.



Rysunek 5: Mapa trapezowa wygenerowana dla zbioru A

4.2.2. Zbiór B

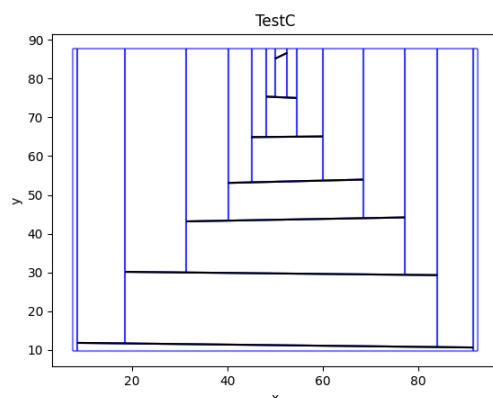
Zbiór ten powstał, by sprawdzić jak program radzi sobie z gęsto rozłożonymi, położonymi blisko siebie odcinkami oraz trapezami.



Rysunek 6: Mapa trapezowa wygenerowana dla zbioru B

4.2.3. Zbiór C

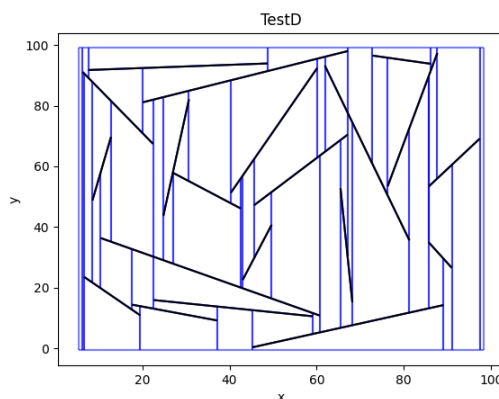
Zbiór ten został przygotowany w celu sprawdzenia czy algorytm poprawnie tworzy trapezy, gdy każdy odcinek zawiera się całkowicie w jednym trapezie.



Rysunek 7: Mapa trapezowa wygenerowana dla zbioru C

4.2.4. Zbiór D

Ostatni ze zbiorów został wygenerowany losowo, sprawdza większą liczbę odcinków oraz przypadki, gdy odcinki zawierają się zarówno w jednym trapezie jak i wielu.



Rysunek 8: Mapa trapezowa wygenerowana dla zbioru D

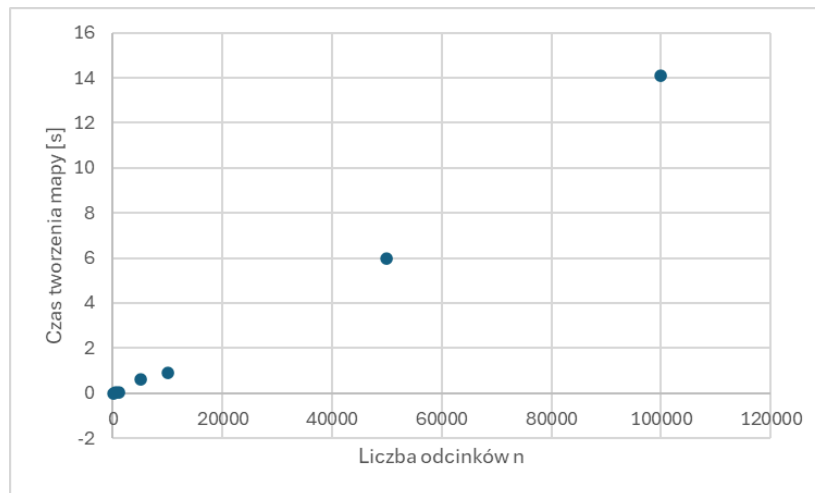
4.3. Testy wydajnościowe

W celu zbadania poprawności i efektywności zaimplementowanego algorytmu przygotowana została funkcja generująca zbiór n losowych odcinków. Została ona wykorzystana w ramach testów czasowych. Testy były powtarzane 10 razy dla tych samych losowych danych, a ostatecznie brana była średnia z otrzymanych wyników. W ten sposób zbadany został czas algorytmu tworzenia mapy trapezowej, czas lokalizacji punktu w przestrzeni oraz rozmiar struktury przeszukiwań.

4.3.1. Czas algorytmu tworzenia mapy trapezowej

	100	500	1000	5000	10000	50000	100000
1	0,004	0,017	0,027	2,818	0,796	5,012	13,222
2	0,003	0,018	0,029	0,379	0,895	6,285	13,685
3	0,004	0,019	0,029	0,395	0,983	6,320	14,611
4	0,003	0,017	0,027	0,349	0,908	6,188	14,293
5	0,003	0,018	0,027	0,347	0,958	6,336	14,427
6	0,003	0,018	0,028	0,316	0,931	5,881	14,094
7	0,003	0,019	0,027	0,301	0,947	5,889	13,096
8	0,004	0,017	0,031	0,357	0,833	5,771	13,998
9	0,004	0,015	0,030	0,339	0,898	5,939	14,457
10	0,003	0,014	0,028	0,334	0,917	6,372	15,056
Średnia	0,003	0,017	0,028	0,594	0,907	6,000	14,094

Tabela 1: Czasy wykonania względem rosnącej liczby odcinków dla kolejnych prób



Rysunek 9: Wykres zależności czasu tworzenia mapy trapezowej od liczby odcinków.

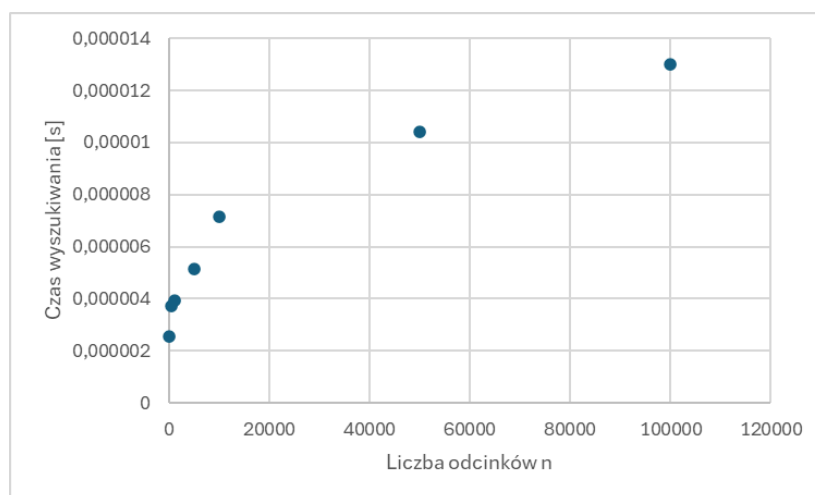
Jak dostrzec można na wykresie przedstawionym na Rys. 9 oraz Tabeli 1, złożoność czasowa algorytmu tworzenia mapy jest zgodnie z oczekiwaniami zbliżona do złożoności rzędu $O(n \log n)$.

4.3.2. Czas lokalizacji punktu na mapie

W celu precyzyjniejszego zbadania zależności czasu lokalizacji punktu na mapie od liczby odcinków, przy każdej próbie lokalizacji dokonano 1000 losowań punktu i policzono średni czas lokalizacji jednego punktu.

	100	500	1000	5000	10000	50000	100000
1	2,51447E-06	3,6617E-06	3,9011E-06	5,03231E-06	7,18005E-06	9,84931E-06	1,3478E-05
2	2,68091E-06	3,99635E-06	3,93629E-06	5,03507E-06	6,89323E-06	1,1039E-05	1,27492E-05
3	2,47979E-06	3,55785E-06	3,75324E-06	5,01292E-06	6,80334E-06	1,10126E-05	1,34281E-05
4	2,42894E-06	3,76369E-06	3,72453E-06	5,22752E-06	7,89546E-06	1,04528E-05	1,62951E-05
5	2,62208E-06	3,82634E-06	3,82059E-06	4,82686E-06	8,5079E-06	9,68108E-06	1,19596E-05
6	2,56804E-06	3,85053E-06	4,07967E-06	5,12897E-06	6,98312E-06	1,03955E-05	1,23524E-05
7	2,791E-06	3,90812E-06	4,00237E-06	5,03228E-06	7,48516E-06	1,02784E-05	1,18726E-05
8	2,59541E-06	3,75348E-06	4,19433E-06	5,33312E-06	6,371E-06	1,08921E-05	1,24382E-05
9	2,47402E-06	3,51259E-06	4,0096E-06	5,22721E-06	6,45572E-06	1,04439E-05	1,29088E-05
10	2,56649E-06	3,41642E-06	3,92091E-06	5,78491E-06	6,84393E-06	1,01726E-05	1,25116E-05
Średnia	2,57212E-06	3,72471E-06	3,93426E-06	5,16412E-06	7,14189E-06	1,04217E-05	1,29994E-05

Tabela 2: Czasy lokalizacji punktu dla różnej liczby odcinków w kolejnych próbach



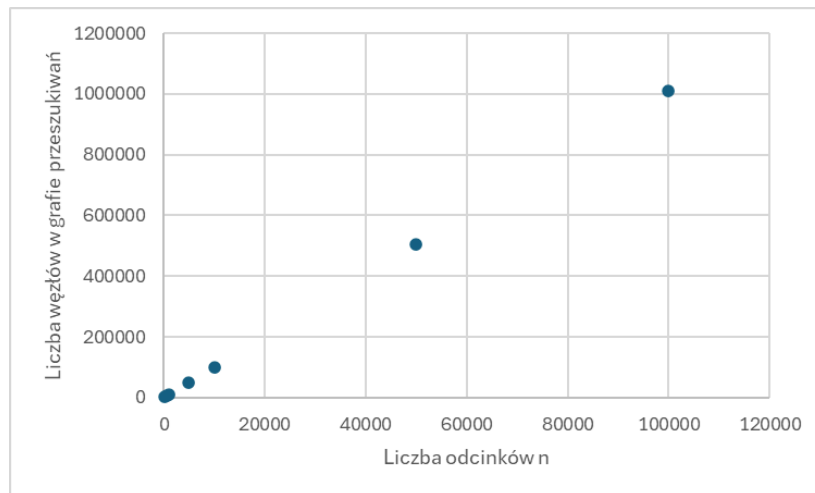
Rysunek 10: Wykres zależności czasu lokalizacji punktu od liczby odcinków.

Oczekiwana złożoność czasowa operacji lokalizacji to $O(\log n)$. Jak ukazują Rysunek 10 oraz Tabela 2, zaimplementowany algorytm wykazuje zgodność z teorią również i w tym wypadku.

4.3.3. Rozmiar struktury przeszukiwań

	100	500	1000	5000	10000	50000	100000
1	935	4890	9615	49651	100243	504637	1010332
2	926	4786	9899	50281	99821	503295	1008995
3	953	4793	9965	49985	100834	503836	1011439
4	890	5040	9684	49711	100301	508156	1013133
5	910	4756	9579	49952	99210	507475	1012926
6	898	4835	9966	49522	100469	503397	1009825
7	966	4908	9481	49829	100717	505726	1014114
8	949	4679	9898	49969	100576	503438	1012511
9	936	4851	10002	49487	100363	505569	1006465
10	917	4896	9682	49491	99868	506872	1014993
Średnia	928	4843,4	9777,1	49787,8	100240,2	505240,1	1011473,3

Tabela 3: Liczba węzłów dla różnej liczby odcinków w kolejnych próbach



Rysunek 11: Wykres zależności liczby węzłów w strukturze przeszukiwań od liczby odcinków.

Jak można bezpośrednio odczytać z danych, proporcja liczby węzłów do liczby odcinków zachowuje niemal stałą proporcję wynoszącą 10:1. Potwierdza to teoretyczną złożoność pamięciową $O(n)$ tej struktury.

5. Podsumowanie

W ramach projektu wykonany został algorytm lokalizacji punktu w przestrzeni metodą trapezową. Zaimplementowana funkcja budująca graf przeszukiwań i mapę trapezową przeszła wszystkie wskazane testy. Zaimplementowane zostały też animacje krokowe ukazujące działanie algorytmu dostępne w folderze *res*. Poprawnie działa też interfejs pozwalający na zadawanie własnych zbiorów odcinków, losowanie ich oraz zapis i odczyt. Wykresy zmierzonych wartości pokrywają się z oczekiwanymi wykresami złożoności.

Literatura

M. van de Berg, M. van Kreveld, M. Overmars, and O. Schwarzkopf, Geometria Obliczeniowa - Algorytmy i Zastosowania. WNT 2007